

CS505 HW0: Classification - Written Report

Author: Zhengzheng Tang
BUID: U07312313
Date: February 2026

Task 1: Bag-of-words Classification

Q1 (3 points)

Implementation: The `BoWFeaturizer` class was implemented in `models.py`. The `build_vocab` method tokenizes text using NLTK's `word_tokenize`, converts to lowercase, counts token frequencies, and keeps the top 5,000 most frequent tokens. The `get_feature_vector` method returns a count vector of vocabulary tokens present in the input text.

Results using `python lang_classifier.py --model BOW`:

Metric	Value
Train Accuracy	0.9782
Dev Accuracy	0.7459
Time	~15 seconds

Q2 (1 point)

Why are training and development accuracies different?

The training accuracy (0.9782) is significantly higher than the development accuracy (0.7459) due to **overfitting**. The model memorizes patterns specific to the training data, including noise and idiosyncrasies that don't generalize to unseen examples. The development set contains different examples that the model hasn't seen during training, so it performs worse. This gap indicates the model has learned some spurious correlations rather than truly generalizable features.

Q3 (2 points)

Top-5 weight indices and corresponding tokens for each class:

After training the BOW model, I examined the weight matrix (shape: $4 \times V$) to find the highest-weighted tokens for each class:

Class	Label	Top-5 Tokens
0	World	'...', '-', 'minister', 'afp', 'president'
1	Sports	'team', 'saturday', 'sunday', 'night', 'for'
2	Business	'company', 'oil', 'business', 'reuters', 'inc.'
3	Tech/Science	'ap', 'scientists', 'space', 'space.com', '\'

Analysis:

- **World news** features political terms ('minister', 'president') and news agency identifier ('afp' - Agence France-Presse). The punctuation tokens ('...', '-') may appear frequently in international news formatting.
- **Sports** contains time-related words ('saturday', 'sunday', 'night') indicating game schedules, and 'team' is directly sports-related.
- **Business** shows corporate terms ('company', 'inc.') and economic indicators ('oil', 'business'), plus 'reuters' as a major financial news source.
- **Tech/Science** includes scientific terms ('scientists', 'space') and domain-specific sources ('space.com', 'ap').

The model has learned meaningful discriminative features. Some tokens like punctuation or common words ('for') may have high weights due to their frequency patterns across classes rather than semantic meaning.

Task 2: Logistic Regression

Q4 (10 points)

Implementation: The `LogisticRegressionClassifier` class was implemented with:

- `forward`: Computes $z = W^T * x + b$ using `torch.matmul`
- `softmax`: Implements numerically stable softmax by subtracting max before exponentiating
- `train_logistic_regression`: Implements SGD with manual gradient computation

Results using `python lang_classifier.py --model LR`:

Metric	Value
--------	-------

metric	value
Train Accuracy	0.9218
Dev Accuracy	0.7529
Time	~5 seconds

Q5 (3 points)

Hyperparameter Tuning Results:

vocab_size	lr	epochs	Dev Accuracy
5000	0.01	10	0.7529
5000	0.001	10	0.6847
5000	0.1	10	0.6200
10000	0.01	20	0.7694
10000	0.01	30	0.7706

Best configuration: vocab_size=10000, lr=0.01, epochs=30, Dev Accuracy=0.7706

Learning Rate Analysis:

- **Too high (lr=0.1):** The model oscillates and fails to converge to a good solution. The loss fluctuates wildly, and final accuracy drops to ~0.62. This happens because large gradient steps overshoot the optimal parameters, causing the optimization to bounce around or diverge.
- **Too low (lr=0.001):** The model converges very slowly and may not reach the optimal solution within the given epochs. Final accuracy is lower (~0.68) because the model hasn't had enough effective updates to learn the decision boundary properly.

Task 3: Feature Engineering

Q6a (3 points)

Bigram Implementation: The `BigramFeaturizer` class generates consecutive word pairs (e.g., "the|cat") and builds a vocabulary of the most frequent bigrams.

Results using `python lang_classifier.py --model BIGRAM:`

--	--

Metric	Value
Train Accuracy	0.9594
Dev Accuracy	0.6718
Dev Macro-F1	0.4870

Q6b (2 points)

Why does the bigram model perform worse despite being more expressive?

The bigram model suffers from **feature sparsity** leading to **overfitting**:

1. **Exponential feature space:** While unigrams have ~10K unique tokens, bigrams have potentially $10K \times 10K$ combinations. Most bigrams appear only once or twice in the training data.
2. **Sparse observations:** With limited training data (1,650 examples), most bigram features have very few observations. The model learns weights for bigrams that happened to appear in training but won't generalize.
3. **Overfitting to noise:** The high train accuracy (0.9594) but lower dev accuracy (0.6718) confirms overfitting. The model memorizes training-specific bigram patterns rather than learning generalizable features.

As discussed in the textbook (Chapter 3.5), more complex models with more parameters require more training data to estimate reliably. With insufficient data, simpler models (unigrams) generalize better.

Q7a (8 points)

Custom Feature 1: TF-IDF Featurizer

Location: `models.py`, line 105, class `TFIDFFeaturizer`

Description: TF-IDF (Term Frequency-Inverse Document Frequency) weighs terms by their importance. Common words get lower weights (low IDF), while rare but discriminative words get higher weights.

Implementation details:

- $TF = \text{count}(\text{word}) / \text{total_words_in_document}$
- $IDF = \log(N / (df + 1)) + 1$ (smoothed)
- Final vector is L2 normalized

Results:

Metric	Value
--------	-------

Dev Accuracy	0.7706
Dev Macro-F1	0.6638

Custom Feature 2: Combined Featurizer (TF-IDF + Bigram)

Location: `models.py`, line 197, class `CombinedFeaturizer`

Description: Concatenates TF-IDF unigram features with bigram features to capture both single-word importance and word-pair patterns.

Results:

Metric	Value
Dev Accuracy	0.7118
Dev Macro-F1	0.5585

Q7b (1 point)

Performance Analysis:

- **TF-IDF outperforms BoW** because it down-weights common words (like "the", "is") that appear in all classes and up-weights discriminative terms. This reduces noise in the feature representation.
- **Combined featurizer underperforms** because the bigram component introduces sparsity and noise that outweighs its benefits. The concatenation also increases dimensionality without proportional information gain, leading to overfitting on the small dataset.

Task 4: A Better Evaluation Metric

Q8a (3 points)

Macro-F1 Implementation: Implemented in `utils.py`. For each class, compute precision, recall, and F1, then average across all classes.

Results for LR model (Q4):

Metric	Value
--------	-------

metric	value
Dev Accuracy	0.7529
Dev Macro-F1	0.6476

Q8b (2 points)

Is F1 significantly different from accuracy?

Yes, the Macro-F1 (0.6476) is noticeably lower than accuracy (0.7529), a difference of about 10%.

What does this tell us about the classifier?

This discrepancy reveals that the classifier performs **unevenly across classes**. The dataset is highly imbalanced (1000 world news vs. 50 sports), so:

- Accuracy is dominated by majority classes:** Getting "world news" right contributes heavily to accuracy since it's 60% of the data.
- Macro-F1 weights all classes equally:** Poor performance on minority classes (sports, tech) significantly hurts the macro-F1 score.
- The classifier likely under-predicts minority classes:** It may predict "world news" too often because that minimizes overall error, but this strategy fails for balanced evaluation metrics.

This demonstrates why Macro-F1 is a better metric for imbalanced datasets—it ensures the model must perform well on ALL classes, not just the frequent ones.

Appendix: Optimization Experiments

Overview

Through systematic experimentation, I improved the Macro-F1 score from **0.5127** (baseline) to **0.7132** (optimized), a **+39.1% improvement**.

Optimization Path

1	Baseline → +Class Weight → +TF-IDF → +L2+Power → +Stopwords+Stem+LRDecay+EarlyStop					
2	0.5127	→	0.5809	→	0.6587 → 0.6777	→ 0.7132
3			(+13.3%)		(+28.5%) (+32.2%)	(+39.1%)

Key Experiments

Experiment 1: Class Weighting

Imbalanced data (sports: 50 samples vs world: 1000 samples) causes the model to ignore minority classes. Class weighting addresses this:

```
1 | weight[c] = (total / (num_classes * count[c])) ^ power
```

Power	Sports Weight	Dev Macro-F1
1.0	8.25	0.6638
1.05	~9.0	0.6777
1.1	~10.0	0.6601

Finding: power=1.05 is optimal; higher values cause overfitting to minority classes.

Experiment 2: TF-IDF Features

TF-IDF improved Macro-F1 by ~8% over BoW by emphasizing discriminative terms.

Experiment 3: L2 Regularization

l2_lambda	Dev Macro-F1
0	0.6626
0.0001	0.6638
0.001	0.6574

Finding: Light regularization (0.0001) helps; too strong causes underfitting.

Experiment 4: Text Preprocessing

Added stopwords filtering and Porter stemming:

Configuration	Vocab Size	Dev Macro-F1
Original	8876	0.6777
+Stopwords+Stemming	5595	0.7110

Finding: Preprocessing reduces noise and feature sparsity.

Experiment 5: Learning Rate Decay

```
1 | current_lr = lr * (decay_rate ^ epoch)
```

lr_decay	Dev Macro-F1
1.0	0.6777
0.95	0.7110
0.90	0.6945

Finding: Gradual LR decay (0.95) improves convergence stability.

Experiment 6: Early Stopping

Monitor dev F1 each epoch; stop if no improvement for `patience` epochs; restore best model.

patience	Dev Macro-F1
3	0.6923
5	0.7045
7	0.7110

Final Best Configuration

```
1 | python lang_classifier.py --model BEST
```

Parameters:

- vocab_size: 10000
- lr: 0.01
- epochs: 50 (early stopping)
- l2: 0.0001
- class_weight_power: 1.05
- lr_decay: 0.95
- patience: 7
- stopwords: enabled
- stemming: enabled

Multi-Seed Validation

Seed	Dev Macro-F1
42	0.7110
123	0.7154
456	0.7163
789	0.7140
999	0.7095
Average	0.7132

Failed Attempts

Method	Result	Reason
EDA Data Augmentation	-1%	Synthetic data introduces noise
Bigram Features	-10%	Feature sparsity causes overfitting
High Learning Rate (0.02)	-2%	Unstable convergence
Strong L2 (0.001)	-1%	Underfitting
class_weight_power > 1.1	-2~10%	Over-emphasis on minority classes

Key Findings

1. **Class weighting is crucial** for imbalanced datasets
2. **TF-IDF significantly outperforms BoW** by reducing common word noise
3. **Stopwords + Stemming** reduce feature sparsity effectively
4. **Learning rate decay** helps stable convergence
5. **Early stopping** prevents overfitting and auto-selects best model
6. **Data augmentation is unnecessary** when class weighting is used properly